

Fast API Questions and Answers

1. What is CRUD in FastAPI?

Ans: In FastAPI, CRUD is short form for Create Read Update and Delete.

Create- It is basically to create an app using POST method.

Read- It is to retrieve/access the elements/data from an app using GET method.

Update- It is to modify the existing data in an app using PUT method.

Delete- It is to delete/remove any data from the app using DELETE method.

2. Why do we use the POST method for inserting data?

Ans: In FastAPI, the POST method have tight integration with Pydantic to create models. Which ensures two main things. Data validation and Auto documentation.

3. Difference between PUT and PATCH

Ans: Both methods are used to update/modify the existing data from an application.

PUT	PATCH
We need to send all the fields along with the one which need to be updated.	We can send only the field need to be changed.
If we miss any field it might get deleted.	Leaves the missing fields as they are.
Suitable for the Pydantic models having mandatory fields.	Suitable for models having Optional fields.

4. What is the path parameter?

Ans: Parameters are the values which we pass. Path parameter is the value which we pass/use in the link/url which represent some location or resource which we need to access. It is often enclosed with in curly braces { } like /todos/{id}.

5. What is the request body in FastAPI?

Ans: Basically a request body is a request or data sent by client/users through mobiles or browsers to the server. Like name, number, country, change password etc... Used to send more data/details than what an URL can handle like methods (POST, PUT...).

6. Why do we use Pydantic models?

Ans: We can say Pydantic model as Brain behind FastAPI. It handles all the data. It is primarily used for data validation, ensure type safety and generate interactive documentation without writing extra code. And also automatically parses JSON data into python object.

7. What status code is returned for successful deletion?

Ans: 204 No Content: Deleted and no data to return.

200 OK: Deletes and sends confirmation or a copy of what has been deleted.

202 Accepted: Delete request accepted, actual process is running in background.

8. How can we check whether a student exists before updating?

Ans: Steps to follow.

1. Using student ID, check in the database for existence.
2. If not found, raise an Exception.
3. Proceed with update.

Example code:

```
@app.patch("/students/{student_id}") # Path Parameter
async def update_student(student_id: int, student_data: StudentUpdate, db: Session = Depends(get_db)):
    # 1. Check if exists
    db_student = db.query(Student).filter(Student.id == student_id).first()

    if not db_student:
        # 2. If not found, raise 404
        raise HTTPException(status_code=404, detail="Student not found")

    # 3. Update if found
    for key, value in student_data.model_dump(exclude_unset=True).items():
        setattr(db_student, key, value)

    db.commit()
    return db_student
```

9. What happens if duplicate IDs are inserted?

Ans: If student ID is the primary key, following errors will occur.

Database Error: The database will reject the insertion and throw **Integrity Error**.

FastAPI Response: If we don't handle this error, FastAPI will return **500 Internal Server Error**.

Best Practice: We should catch this error in code and return a **400 Bad Request** or **409 Conflict** status code to the user, explaining that the ID already exists.

10. How does FastAPI automatically generate Swagger UI?

Ans: FastAPI follows these steps in the background.

Code Inspection: When we start the app FastAPI scans the code first. It looks the code for type hints, Pydantic models, and path operations (like `@app.post()`).

OpenAPI Schema Generation: Using that information, it creates a large JSON object called an OpenAPI schema. This schema is a standardized "map" of our entire API, including every endpoint, required parameter, and data type.

The /openapi.json Route: FastAPI automatically registers a hidden route at `/openapi.json` that serves this generated schema.

Swagger UI Rendering: When we visit `/docs`, FastAPI serves a small HTML page that includes the Swagger UI JavaScript. This JavaScript library reads the JSON from `/openapi.json` and renders it into the interactive interface we see in our browser.